

Discrete Mathematics

Graph Theory



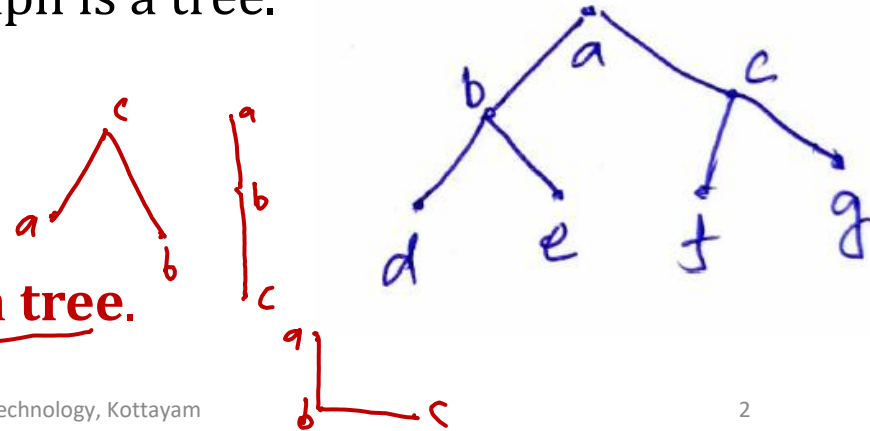
Trees

- **Trees** are the most important class of graphs and they make fine modeling tools.
- Trees facilitate sorting and searching, representing and computing numeric expressions, and constructing efficient coding schemes and algorithms.
- Notice the following connected graph, which is acyclic, meaning it does not contain a cycle. Such a graph is a tree.

- **Definition:**

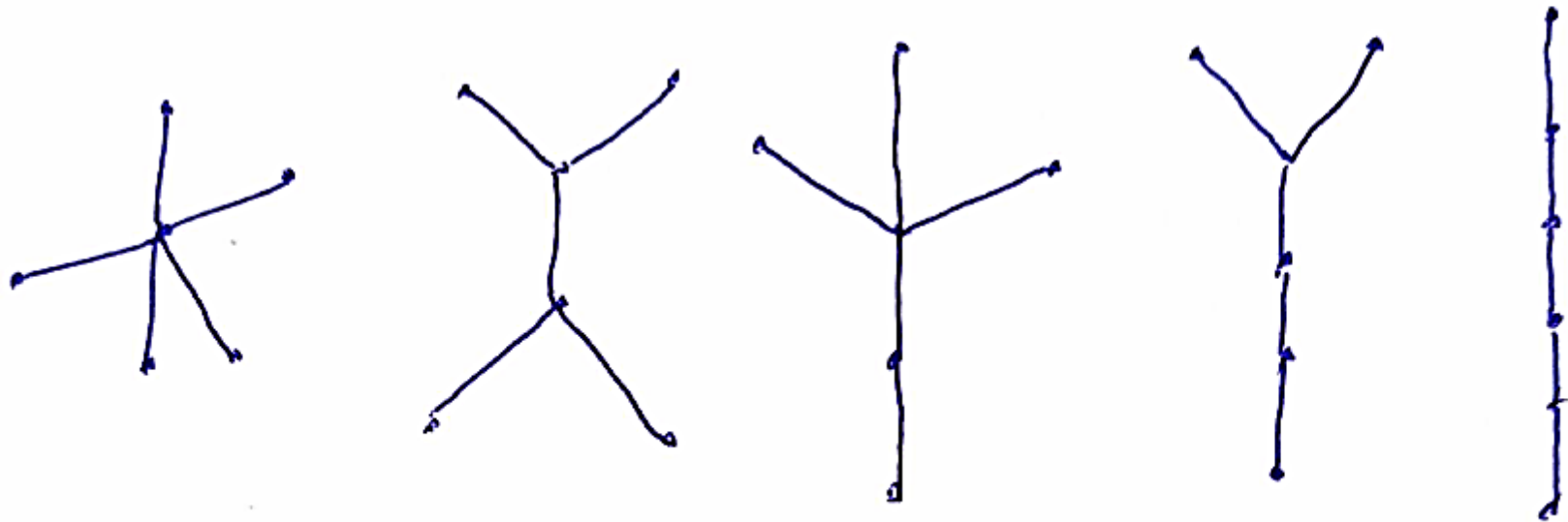
A graph that contains no cycles is called an **acyclic graph**.

A connected acyclic graph is called a **tree**.

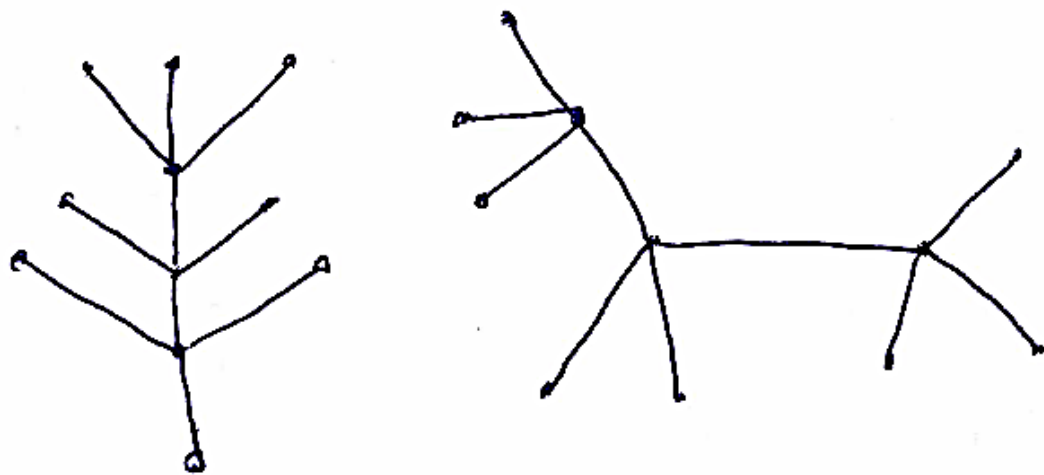


Note A set of disjoint trees, called a ~~forest~~ forest

Draw the trees with 6 vertices



Note Any graph without cycles is also called a forest so that the components of a forest are trees.



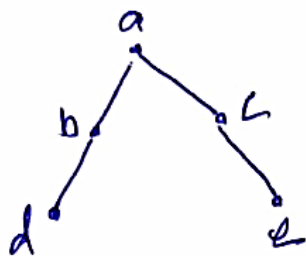
Forest

Note

1. A connected graph is a tree $\Leftrightarrow$ there is a unique, simple path between any two vertices.
2. A connected graph with n vertices is a tree $\Leftrightarrow$ it has exactly $n-1$ edges.

Rooted Trees

A Tree with a root is a rooted tree



Tree rooted at a

Let T be a rooted tree with root v_0

Let $v_0, v_1, v_2, \dots, v_{n-1}, v_n$ be the path from v_0 to v_n
then

- v_{n-1} is the parent of v_n
- v_n is a child of v_{n-1}

- vertices with the same parent are siblings
- The vertices $v_0, v_1, \dots, v_{n-1}$ are ancestors of v_n
- The descendants of a vertex v are those vertices for which v is an ancestor.
 ie [A vertex is called descendant of another ~~node~~ ^{vertex} if it is the child of that vertex]
 ie all the vertices in the tree are descendant of the root.
- A vertex with no children is a leaf or a terminal vertex.
- the subtree of T rooted at v consists of v , its descendants, and all edges incident with them.

level

level of a vertex v in a rooted tree, denoted by $\text{level}(v)$, is the length of the path from the root to v

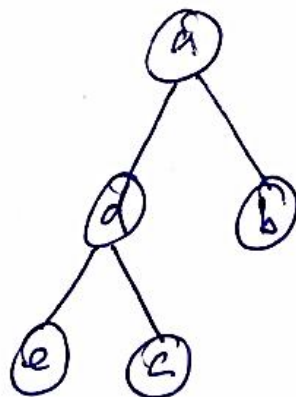
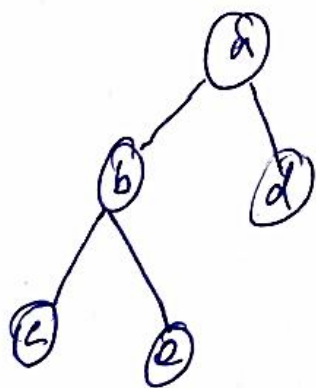
Height

The height of a rooted tree is the maximum level of any leaf in the tree

Note The path length of a vertex in a rooted tree is defined to be the number of edges in the path from the root to the vertex.

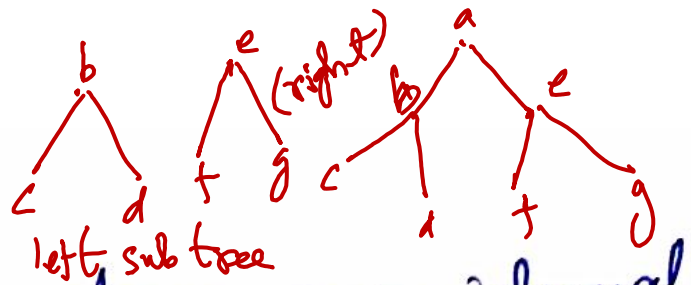
ordered Trees

If in a tree at each level, an ordering is defined, then such a tree is called an ordered tree.



A rooted tree is an m-ary tree if every vertex has at most m children. If $m=2$, it is a binary tree

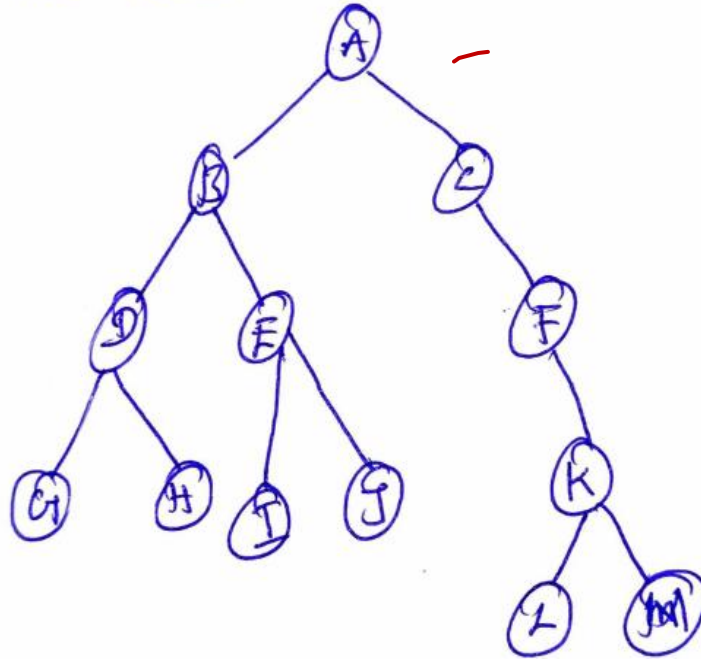
Binary Trees



A binary tree is a 2 ary tree, every internal vertex has atmost two children, the elder is the left child, the other is the right child

The subtree at a left child v is the left subtree rooted at v and the subtree rooted at a right child w is the right subtree rooted at w

1 For the tree



Find

(i) The children of each node

(ii) Siblings

(iii) level of each node

(iv) Height of the tree

(v) left and right subtree of node A

tree traversals

An important tree operation is tree traversal visiting every vertex of a tree in a systematic way. Three methods for traversing a nonempty binary tree.

preorder

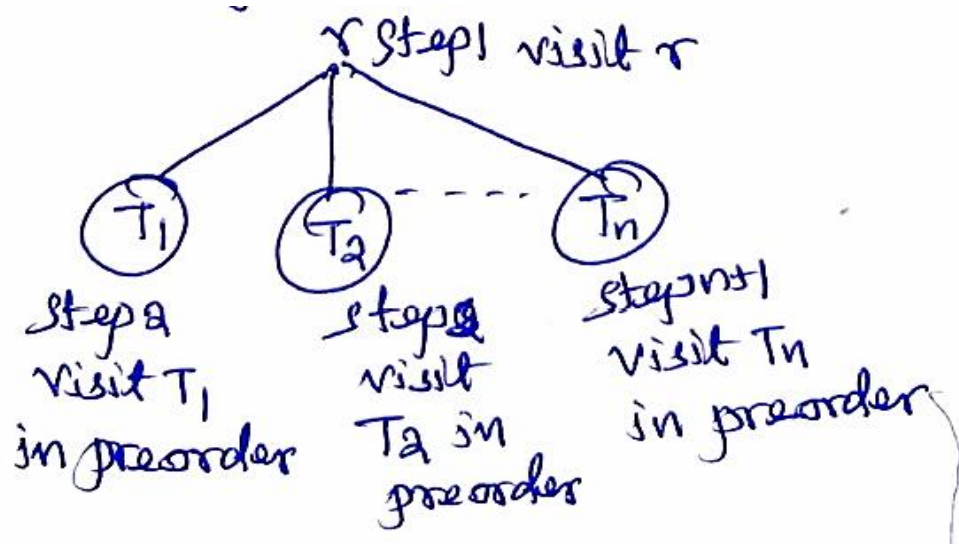
Inorder

postorder

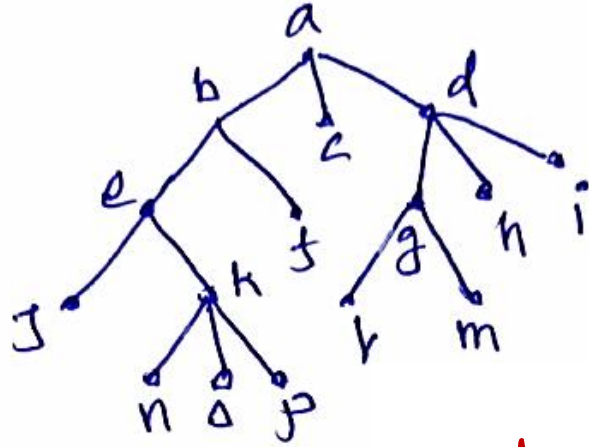
Each of them can be defined recursively

preorder traversal

1. visit the root
2. Traverse the left subtree in pre order
3. Traverse the right subtree in pre order



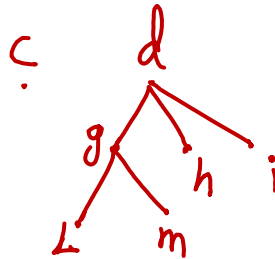
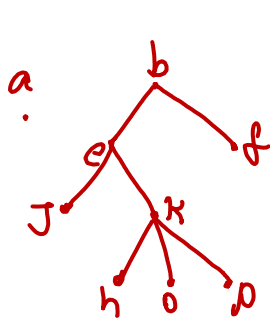
Q write the preorder Traversal of the tree



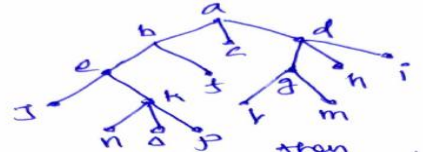
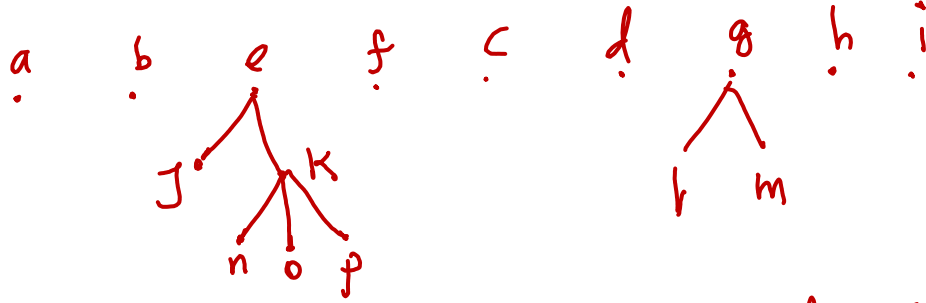
Soln -

visit root of Tree $\xrightarrow{\text{then}}$ visit left Subtrees $\rightarrow$ Right Subtrees

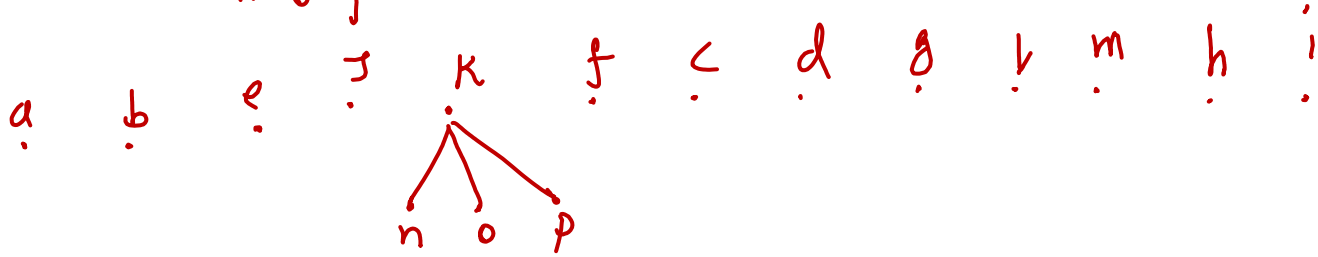
Step 1



step 2



Step 3



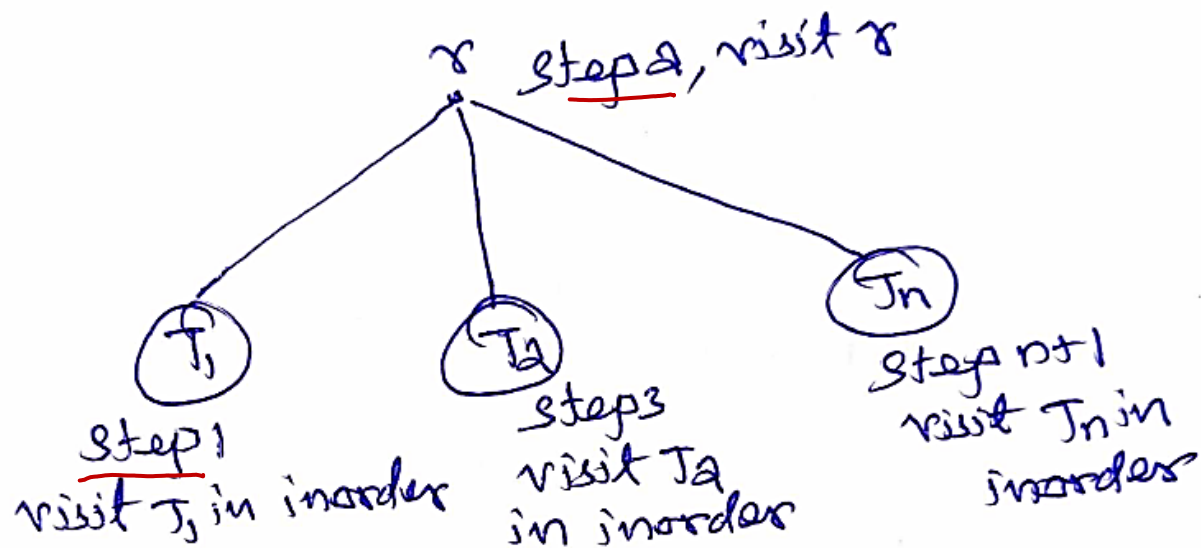
Step 4



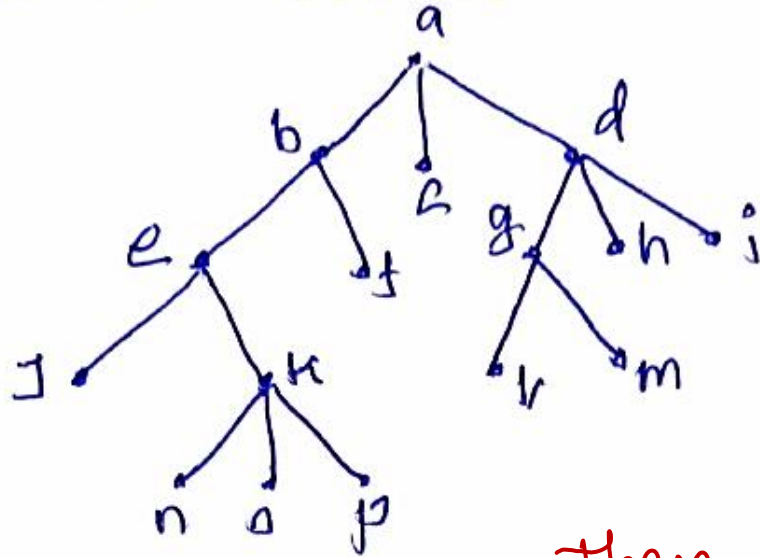
This is preorder traversal of given Tree

2. the inorder traversal of a tree is

- (i) Traverse in inorder the left subtree
- (ii) Visit the root of the tree
- (iii) Traverse in inorder the right subtree



write the inorder Traversal of the tree



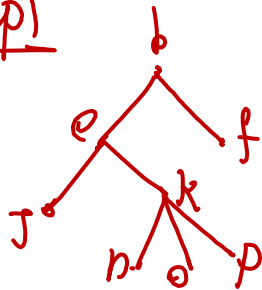
step 3

j e k b f a c i g m d h i
 n o p

step 4

j e n k o p b f a c i g m
 This is inorder traversal of given Tree
 visit

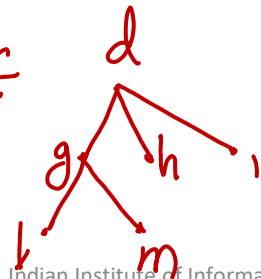
Soln
step 1



visit left subtrees

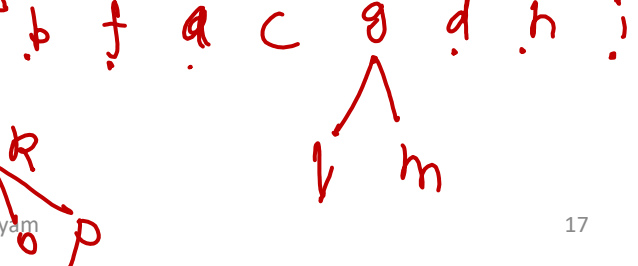
Then

step 2

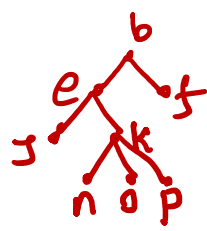


root

visit right
 Subtrees

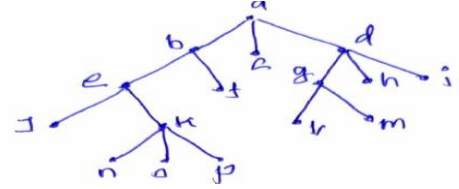


Step 1:



a .

c .



Step 2:



b .

f .

a .

c .

g .

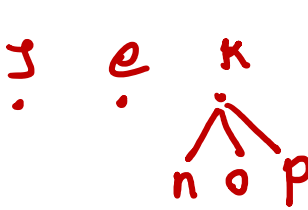
d .

h .

i .



Step 3:



b .

f .

a .

c .

v .

g .

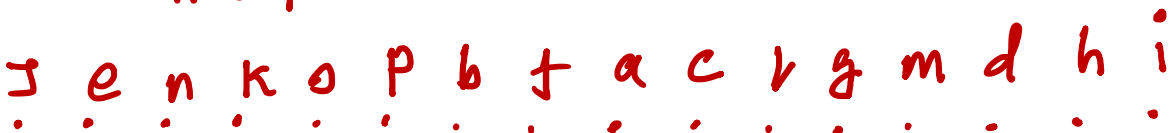
m .

d .

h .

i .

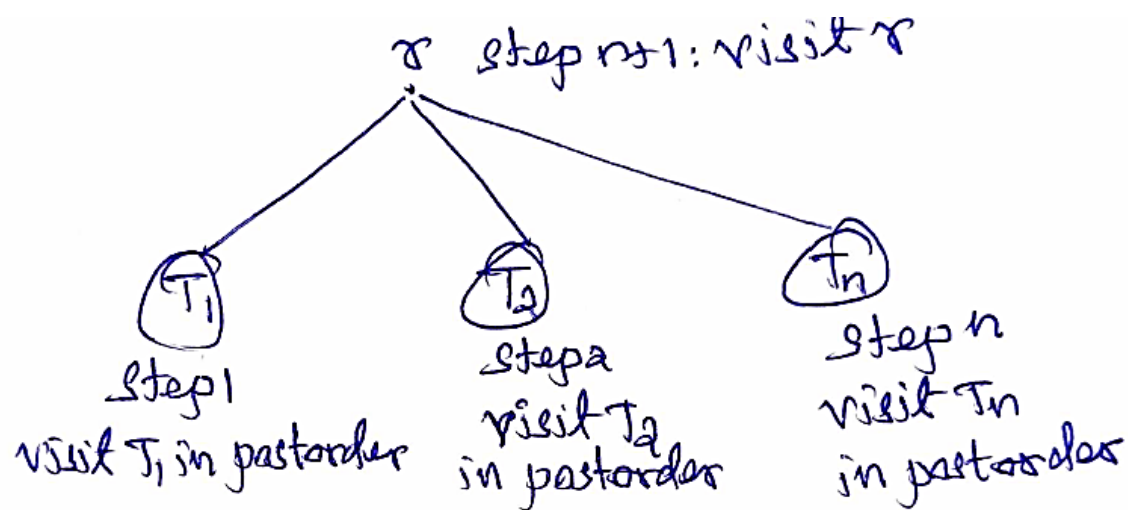
Step 4:



This is inorder traversal of the given tree.

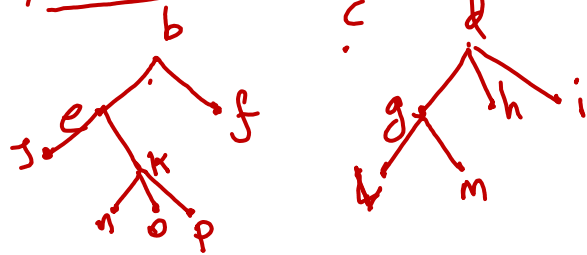
The postorder traversal of a tree is

- (i) Traverse the left subtree in postorder
- (ii) Traverse the right subtree in postorder
- (iii) visit the root of the tree

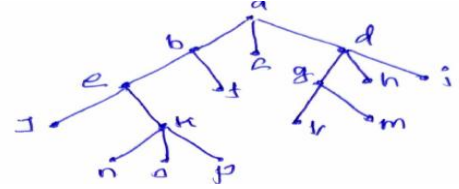


post-order Traversal

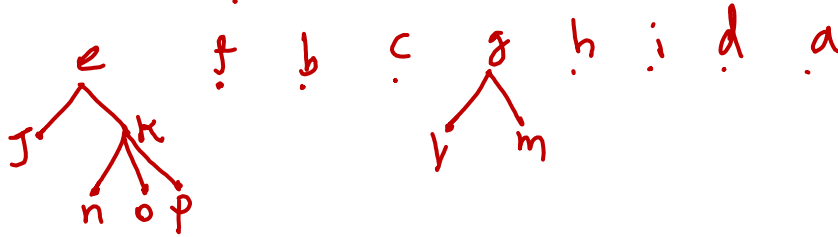
Step 1:



a



Step 2:



Step 3:

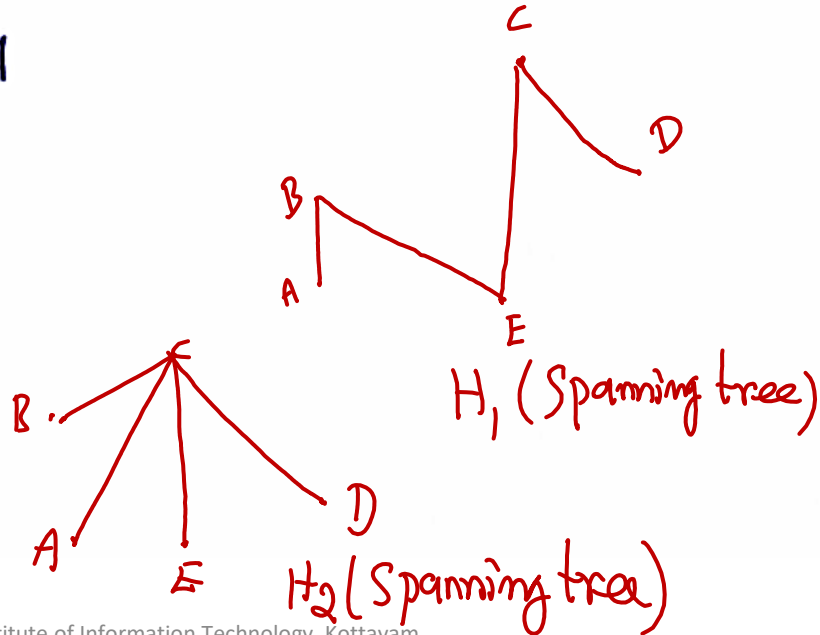
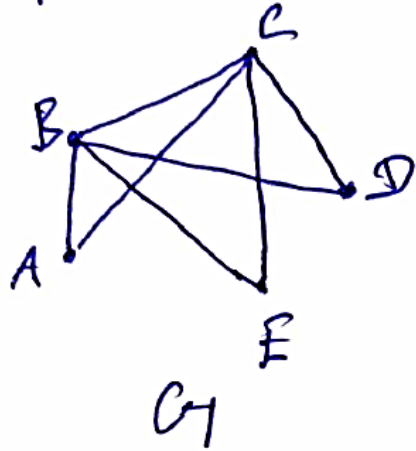


Step 4:

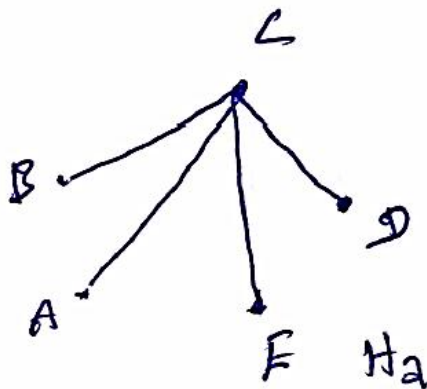
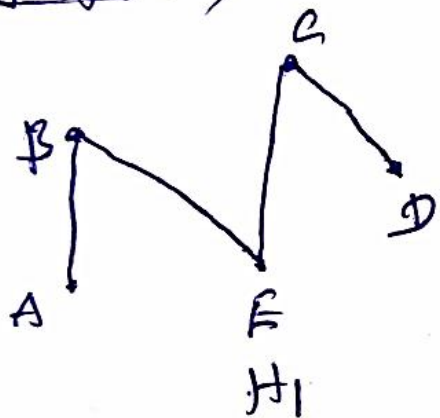


Spanning tree

A subgraph H of a connected graph G is a spanning tree of G if H is a tree containing every vertex of G .



Subgraph(H)



here H_1 and H_2 are the spanning tree of G

Note: The spanning tree of a graph need not be

1. Unique

✓ 2. Every connected graph has a spanning tree

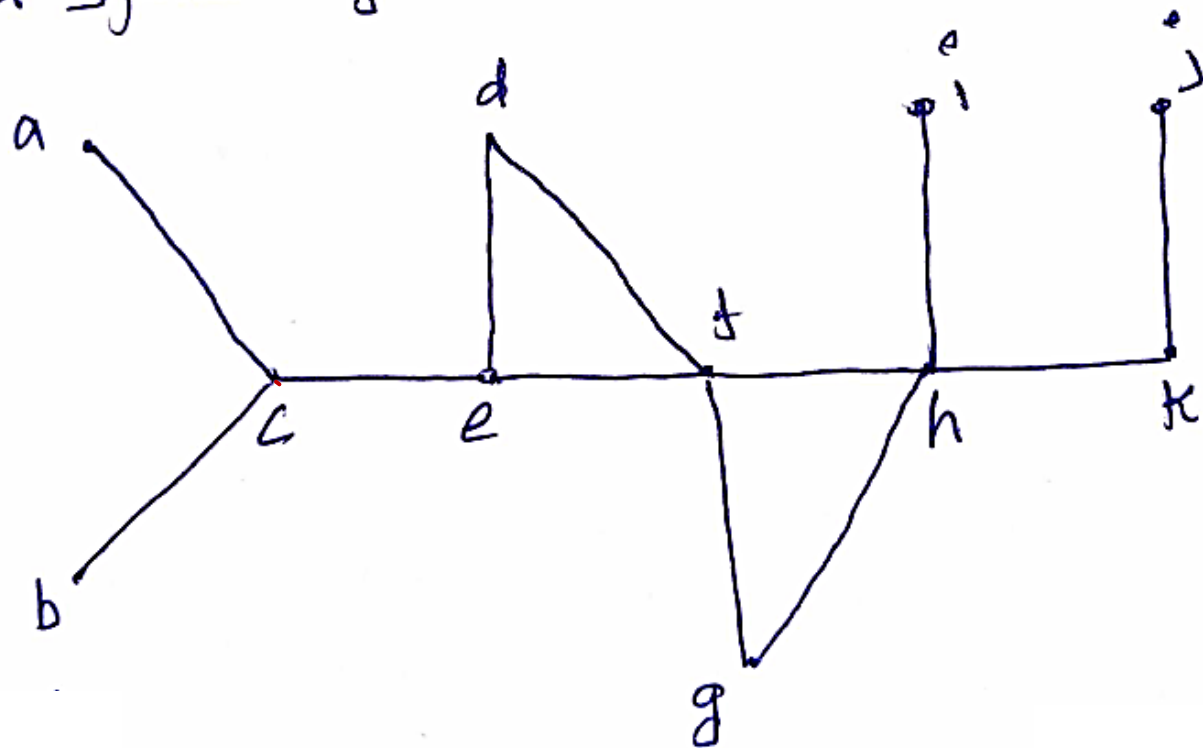
The Depth-First Search (DFS) Algorithm

For the DFS algorithm, beginning at a vertex u , traverse the graph as far as possible, marking each vertex visited. Once you reach a vertex from which no ~~no~~ more paths are available, backtrack to the last vertex visited; continue traversing in this manner until all vertices are visited.

Outline of DFS algorithm

1. Start at a vertex u
2. Mark it as visited
3. pick a vertex v adjacent to u and not yet visited
4. call DFS from v
5. If a vertex w is reached such that all its adjacent vertices have been visited, back up to the last vertex x visited and call DFS from x
6. stop when you have visited all vertices.

use depth first search algorithm to find a spanning tree of the following graph



Soln

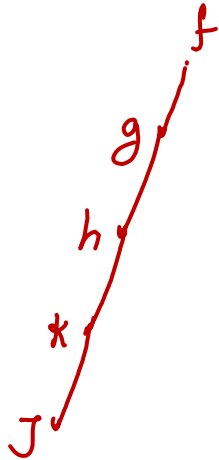
Note: Depth or height of a tree is the maximum level of any leaf in the tree.

level: length of the path from the root to v

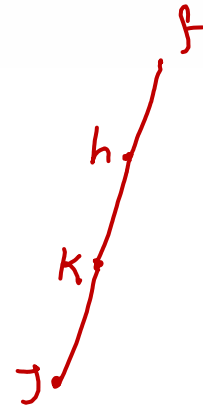
Step 1: we arbitrarily start with vertex f
[maximum level from f to leaf node)
is 4

f

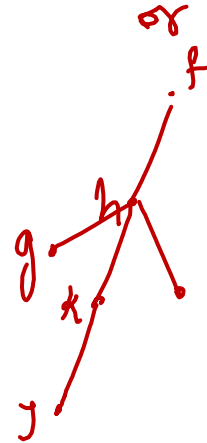
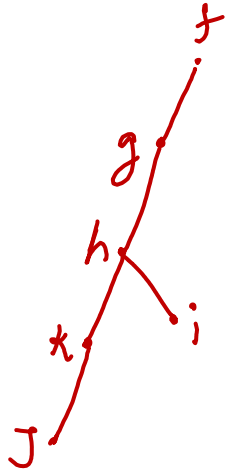
Step 2: Form a path by successively adding edges with non-repeated vertices in the path as long as this possible.



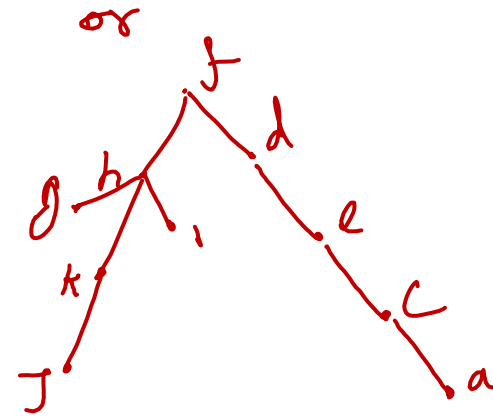
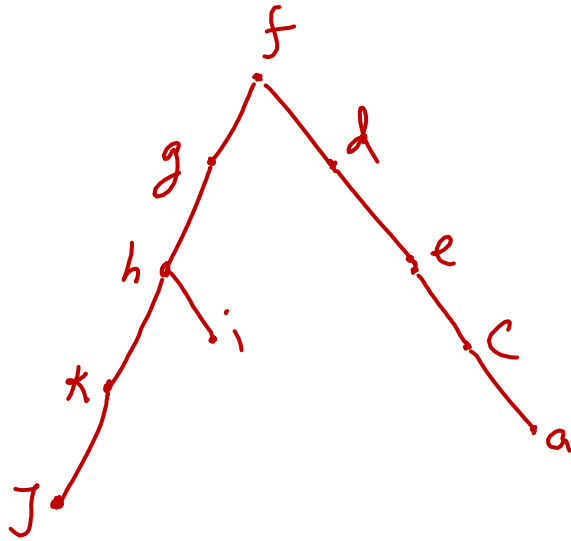
or



Step 3: backtrack to k , there is no path
beginning at k containing new vertex
So backtrack to h , now form a path $h-i$

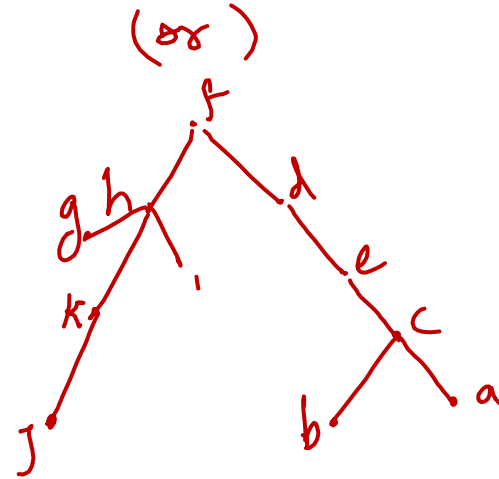
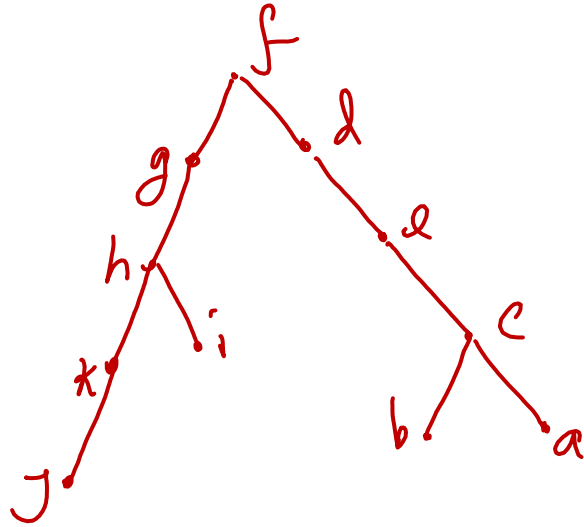


Step 4: then backtrack to h and then f
now form a new path f-d-e-c-a



steps

Then backtrack to e and form the path $e-b$
(now all the vertices are visited, so stop the algorithm)



Find the preorder, postorder and inorder traversal of the tree

